



Unidad Didáctica

Gestión de Calidad de Proyectos de Software, Unidad IV

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	5
Conceptos de calidad	5
Técnicas de revisión	13
Aseguramiento de la calidad	20
Estrategias de pruebas	28
Bibliografía y Webgrafía	29
Glosario	29

Introducción

La calidad del software constituye un requisito crítico en el desarrollo de sistemas contemporáneos, trascendiendo la mera funcionalidad para convertirse en un factor determinante de confiabilidad, seguridad y sostenibilidad tecnológica. Su relevancia se evidencia en incidentes documentados como el fallo del sistema de salud Healthcare.gov (2013), donde deficiencias en pruebas de carga generaron pérdidas estimadas en USD 840 millones, o la brecha de seguridad de Equifax (2017) que comprometió datos de 147 millones de usuarios. Estos casos demuestran que los errores de software impactan no solo en la operatividad técnica, sino en dimensiones económicas, legales y sociales.

El estudio sistemático de la calidad del software se estructura en cuatro dominios interrelacionados. En primer lugar, los modelos conceptuales como ISO 25010 proporcionan marcos estandarizados para definir y medir atributos de calidad, estableciendo criterios objetivos para evaluar características como seguridad, rendimiento y mantenibilidad. Estos modelos permiten traducir expectativas subjetivas en requisitos verificables.

Posteriormente, las técnicas de revisión formal e informal representan mecanismos preventivos para identificar defectos en etapas tempranas del ciclo de desarrollo. Metodologías como las inspecciones de Fagan y el pair programming han demostrado reducir hasta en un 85% los errores que alcanzan fases de producción, según estudios del Software Engineering Institute (SEI).

El aseguramiento de la calidad (SQA) formaliza estos procesos mediante planes estructurados (SQAP/IEEE 730) y auditorías sistemáticas, integrando controles que garantizan el cumplimiento de estándares y regulaciones. Esta disciplina se potencia mediante herramientas de automatización como SonarQube, que permiten monitorización continua de métricas técnicas.

Finalmente, las estrategias de pruebas abarcan desde la validación unitaria hasta pruebas de seguridad y rendimiento, aplicando técnicas basadas en especificaciones (caja negra) o estructura interna (caja blanca). Este enfoque estratificado mitiga riesgos mediante la detección temprana de vulnerabilidades, como inyecciones SQL o fallos de carga, antes de su despliegue en entornos productivos.

La convergencia de estos dominios responde a un imperativo económico: según datos de IBM, el costo de corregir defectos en producción puede ser 100 veces superior al de su detección en diseño. En contextos regulados como fintech o salud, donde aplican normativas como PCI-DSS o HIPAA, la implementación rigurosa de estos principios constituye no solo una mejor práctica técnica, sino un requisito legal.

Conceptos de calidad

La calidad del software es un concepto complejo y multidimensional que abarca mucho más que el simple cumplimiento de requisitos técnicos o funcionales. Implica evaluar la **capacidad de un producto o sistema para satisfacer tanto los requerimientos explícitos** (como las funcionalidades especificadas por el cliente o usuario final) **como los implícitos** (las expectativas de uso, facilidad, seguridad, entre otros).

Este análisis requiere el uso de **modelos estructurados de calidad**, que descomponen la calidad en atributos y subcaracterísticas medibles. Gracias a estos modelos, las organizaciones pueden monitorear, evaluar y mejorar sus productos de manera continua, fundamentándose en datos y no únicamente en percepciones subjetivas.

Modelos Clave y Ejemplos Detallados

a) ISO/IEC 25010 (2011)

Este estándar internacional define ocho características principales de calidad, cada una subdividida en subcaracterísticas que permiten un análisis más profundo. A continuación, se amplía la explicación de cada característica, incluyendo ejemplos situados en contextos que pueden ser familiares para estudiantes nicaragüenses:

- **Funcionalidad:** Garantiza que el software haga lo que se espera.
 - *Ejemplo:* Un sistema de gestión estudiantil para universidades nicaragüenses que procese correctamente las matrículas sin errores de cálculo (corrección) y contemple todas las modalidades de pago locales (completitud).
- **Fiabilidad:** Capacidad de mantener el rendimiento bajo condiciones específicas.

- *Ejemplo:* Un sistema de monitoreo para estaciones meteorológicas que continúe operando aun si se interrumpe temporalmente la conectividad a internet (tolerancia a fallos).
- **Usabilidad:** Facilidad de aprendizaje, uso y operación.
 - *Ejemplo:* Una aplicación móvil del gobierno para consultas de cédulas que ofrezca tutorial interactivo (aprendizaje) y botones accesibles para personas con discapacidad visual (accesibilidad).
- **Eficiencia:** Uso óptimo de recursos y tiempos de respuesta.
 - *Ejemplo:* Plataforma que permite consultar datos del Instituto Nicaragüense de Estudios Territoriales (INETER) cargando miles de registros en menos de un segundo.
- **Mantenibilidad:** Facilidad para corregir errores, mejorar o adaptar.
 - *Ejemplo:* Un sistema de facturación desarrollado por módulos independientes, de modo que actualizar la normativa fiscal no afecte otras partes del sistema.
- **Portabilidad:** Facilidad para transferir el software a diferentes entornos.
 - *Ejemplo:* Un sistema contable que funcione sin modificaciones tanto en Windows como en Linux.
- **Compatibilidad:** Capacidad de trabajar junto a otros sistemas.
 - *Ejemplo:* Una API que permita integrar datos del Registro Civil con plataformas municipales.
- **Seguridad:** Protección contra accesos no autorizados o alteraciones.
 - *Ejemplo:* Aplicación bancaria nacional que emplea cifrado AES-256 para proteger transacciones.

b) Modelo de McCall (1977)

Pionero en estructurar la calidad del software en tres perspectivas que facilitan su comprensión y medición:

- **Operación del producto:** se centra en cómo el software funciona en su uso diario (corrección, fiabilidad, eficiencia).
- **Revisión del producto:** considera aspectos internos que facilitan su modificación (mantenibilidad, testeabilidad, flexibilidad).
- **Transición del producto:** examina la capacidad de adaptarse o integrarse en nuevos entornos (portabilidad, reusabilidad, interoperabilidad).

Ejemplo: En un sistema para el control aéreo del Aeropuerto Internacional Augusto C. Sandino, la fiabilidad se prioriza claramente sobre la reusabilidad, ya que una falla podría tener consecuencias graves.

c) Modelo de Boehm (1978)

Organiza la calidad en niveles, destacando qué atributos son esenciales:

- **Nivel Primario:** portabilidad, eficiencia y fiabilidad.
- **Nivel Secundario:** mantenibilidad, testeabilidad y usabilidad.

Ejemplo: Un ERP utilizado por empresas exportadoras de café en Nicaragua prioriza fiabilidad (para evitar pérdida de datos de exportación) y mantenibilidad (por los cambios en leyes fiscales).

1.2 Costos de Calidad

Los costos de calidad se refieren a las inversiones y gastos relacionados con evitar, detectar y corregir defectos en un producto. Como afirmaba Philip Crosby: *"Hacerlo bien desde la primera vez reduce los costos totales"*. Este principio cobra especial relevancia en Nicaragua, donde muchas empresas de software tienen presupuestos limitados y un defecto grave puede comprometer su sostenibilidad.

Clasificación según Juran & Gryna (1988)

Tipo de Costo	Descripción	Ejemplo Local	% estimado del total
<i>Prevención</i>	Actividades para evitar defectos.	Talleres de capacitación para programadores, diseño de prototipos.	5–15%
<i>Evaluación</i>	Verificación y validación del producto.	Pruebas funcionales, revisiones de código, revisiones por pares.	20–35%
<i>Fallos Internos</i>	Defectos detectados antes de la entrega.	Retrabajo, depuración en fase de pruebas.	25–40%
<i>Fallos Externos</i>	Defectos descubiertos por el cliente.	Soporte, actualizaciones urgentes, multas por incumplimiento de contratos.	20–40%

Relación costo–beneficio

Crosby demostró que una inversión modesta en prevención puede evitar gastos mucho mayores en correcciones o indemnizaciones.

Ejemplo práctico:

Una empresa que desarrolla un sistema para la administración de hospitales en Nicaragua invierte \$50,000 en pruebas de seguridad y logra evitar un fallo que hubiera costado \$500,000 en multas y soporte.

1.3 Relación Calidad–Proceso

Premisa central

La calidad de un producto depende directamente de la madurez de los procesos que lo producen (Humphrey, 1989). Procesos bien definidos permiten anticipar problemas y reducir defectos.

a) CMMI (Capability Maturity Model Integration)

Modelo que describe cinco niveles de madurez, cada uno asociado a prácticas de gestión y desarrollo:

<i>Nivel</i>	<i>Características principales</i>	<i>Impacto</i>
<i>Inicial (Ad-hoc)</i>	Procesos improvisados.	Alta tasa de defectos.
<i>Gestionado</i>	Se establecen métricas y control.	Reducción de fallos externos en ~40%.
<i>Definido</i>	Procesos documentados y estandarizados.	Mejora en mantenibilidad (ISO 25010).
<i>Gestionado Cuantitativamente</i>	Uso de estadísticas y métricas.	Mayor eficiencia y previsibilidad.
<i>Optimización</i>	Mejora continua e innovación.	Innovación y ventaja competitiva.

Ejemplo: Una empresa nicaragüense que sube de Nivel 1 a Nivel 3 reduce un 30% la cantidad de bugs detectados en producción.

b) ISO 9001

Esta norma de gestión de calidad enfatiza:

- **Enfoque por procesos:** entender y gestionar las actividades como un sistema interconectado.
- **Mejora continua:** mediante el ciclo PDCA (Planificar, Hacer, Verificar, Actuar).
- **Toma de decisiones basada en datos:** utilizar métricas objetivas.

Al certificarse en ISO 9001, una empresa mejora atributos como fiabilidad y mantenibilidad, gracias a procesos más predecibles y mejor controlados.

Autoevaluación y Evaluaciones (Conservadas y Ampliadas)

Subtema 1.1 – Clasificación de atributos ISO 25010

- Actividad: Clasificar 10 atributos o características según el modelo ISO 25010.
- Ejemplo: “Tiempo de respuesta <2 seg” → Eficiencia (Comportamiento temporal).
- *Criterios de evaluación:*
 - Correcta clasificación (70%)
 - Justificación técnica (30%)

Subtema 1.2 – Cálculo de costos de calidad

- Escenario:
 - Prevención: \$20,000
 - Evaluación: \$35,000
 - Fallos internos: \$50,000
 - Fallos externos: \$120,000
- Actividades:
 - Calcular el costo total.
 - Determinar el porcentaje por categoría.
 - Proponer medidas para reducir fallos externos (p.ej., automatizar pruebas o capacitar equipos).
- *Criterios de evaluación:*
 - Cálculos correctos (50%)
 - Análisis (30%)
 - Propuestas (20%)

Subtema 1.3 – Mapa conceptual de CMMI

- Actividad: Crear un mapa conceptual que relacione:
 - Niveles de madurez CMMI.
 - Prácticas asociadas.
 - Atributos de calidad afectados.
- *Criterios de evaluación:*
 - Precisión de relaciones (60%)
 - Profundidad técnica (30%)
 - Claridad gráfica (10%)

Evaluación final Tema 1 – Análisis de caso

- *Caso: Fallo en app de salud, buffer overflow expuso 500,000 registros; costos totales por errores de proceso y seguridad: >\$3.5 millones.*
- Actividades:
 - Identificar atributos vulnerados.
 - Calcular costos de calidad afectados.
 - Relacionar con modelos de proceso.
- *Criterios de evaluación:*
 - Identificación correcta (30%)
 - Análisis económico (30%)
 - Relación con procesos (30%)
 - Claridad (10%)

Técnicas de revisión

En la Ingeniería de Software, las revisiones constituyen un conjunto sistemático de prácticas orientadas a identificar defectos y mejorar la calidad de los artefactos producidos (código fuente, diagramas, documentación, requisitos, etc.). Su importancia radica en que los defectos detectados en fases tempranas —como especificación o diseño— suelen ser mucho menos costosos de corregir que aquellos encontrados en producción (Boehm, 1981).

Por ejemplo, un error en el diagrama de casos de uso para el desarrollo de un sistema de matrículas universitarias en Nicaragua puede evitar múltiples errores posteriores en codificación, ahorrando recursos económicos y tiempo.

2.1 Revisiones Formales

Las revisiones formales se caracterizan por seguir un proceso definido y documentado, contar con roles específicos, y generar métricas que alimentan la mejora continua de procesos. Su objetivo principal es reducir defectos latentes antes de la entrega final.

a) Inspecciones Fagan (Fagan, 1976)

Son una de las técnicas más representativas y rigurosas, estructuradas en seis fases:

1. **Planificación:** El moderador selecciona los revisores y distribuye los materiales.

Ejemplo contextualizado: En una universidad nicaragüense, un equipo revisa el módulo que valida las notas de los estudiantes antes de subirlas al portal académico. *

2. **Reunión de inicio:** El autor explica el propósito y contexto del artefacto.
3. **Preparación individual:** Cada revisor analiza el artefacto usando checklists específicos, por ejemplo:

- Validación de entradas.
- Uso correcto de sentencias SQL.
- Cumplimiento de patrones de diseño.

4. **Reunión de inspección:**

- **Moderador:** Conduce el proceso.
- **Lector:** Expone el artefacto paso a paso.
- **Autor:** Escucha y aclara dudas.
- **Revisores:** Detectan defectos sin proponer soluciones en la reunión (foco exclusivo en identificar, no en resolver).

5. **Retrabajo:** El autor realiza las correcciones.

6. **Seguimiento:** El moderador verifica que los defectos fueron corregidos adecuadamente.

Impacto comprobado: En IBM, las inspecciones Fagan permitieron reducir defectos en un 85% en software crítico para mainframes (Fagan, 1976).

b) STR (Software Technical Review)

Es un proceso de revisión técnica realizada por pares que, aunque formal, es menos rígida que Fagan:

- Enfocada principalmente en:
 - Verificar cumplimiento de estándares específicos, como MISRA-C en sistemas embebidos o ISO/IEC 25010 para calidad.
 - Detectar vulnerabilidades críticas como inyección SQL o XSS.
- Se permite el debate técnico sobre mejoras y soluciones, haciendo el proceso más flexible.

Ejemplo: Revisión del diseño de una API REST que gestionará datos de estudiantes en una universidad local, verificando que cumpla con principios REST y normas de seguridad.

2.2 Revisiones Informales

Son más ágiles, menos costosas y se integran bien en entornos educativos o proyectos pequeños. Aunque menos sistemáticas, siguen siendo muy útiles para detectar defectos evidentes.

a) Walkthroughs

- Son reuniones dirigidas por el autor del artefacto, quien explica el flujo o funcionalidad mientras los demás participantes exploran posibles problemas.
- Suelen usarse para:
 - Validar lógica de algoritmos.
 - Revisar diagramas de flujo o de clases.
- Preguntas típicas:
 - “¿Qué sucede si el usuario ingresa un dato vacío?”
 - “¿Qué pasa si falla la conexión a la base de datos?”

Ventaja: Son muy útiles cuando los requisitos iniciales pueden tener ambigüedades, algo frecuente en sistemas académicos desarrollados a la medida (Pressman, 2020).

b) Pair Programming

Una práctica derivada de metodologías ágiles, como XP (Extreme Programming):

- **Driver:** Escribe el código.
- **Navigator:** Analiza la estrategia, prevé errores, propone mejoras.

Beneficios evidenciados (Williams & Kessler, 2000):

- Reducción de defectos hasta en un 50%.
- Generación de código más mantenible.
- Favorece transferencia de conocimiento entre miembros del equipo.

Ejemplo local: Dos desarrolladores trabajan juntos en la creación de una función para validar pagos en línea en una plataforma de matrícula.

c) Checklists

Son listas de verificación basadas en:

- Errores históricos detectados previamente.
- Normativas o estándares relevantes (ej.: Ley 787 para protección de datos personales en Nicaragua, HIPAA si fuera software médico).

Plantilla simple para revisión de código:

- [] Se valida el tamaño de los datos ingresados.
- [] Se manejan las excepciones correctamente.
- [] Existen comentarios que explican funciones complejas.
- [] La complejidad ciclomática es razonable (< 10).

Ejemplo: Antes de liberar un módulo que exporta reportes de estudiantes, se revisa con un checklist para asegurar que los datos se protegen adecuadamente.

2.3 Métricas de Revisión

Permiten evaluar la efectividad del proceso de revisión y retroalimentar la calidad del equipo (Grady, 1992).

<i>Métrica</i>	<i>Fórmula</i>	<i>Estándar óptimo</i>	<i>Uso práctico</i>
<i>Densidad de defectos</i>	Defectos / KLOC (mil líneas de código)	< 5 defectos/KLOC	Evaluar calidad del módulo
<i>Eficacia</i>	$(\text{Defectos detectados} / \text{Defectos totales}) \times 100$	$> 80\%$	Medir la capacidad de detección
<i>Tiempo por defecto</i>	Horas invertidas / Defectos detectados	< 0.5 horas/defecto	Optimizar esfuerzos
<i>Eficiencia</i>	$\text{Defectos detectados} / \text{Horas invertidas}$	> 4 defectos/hora	Comparar técnicas o equipos

Ejemplo:

- LOC revisadas: 500
- Defectos detectados: 8
- Tiempo: 2 horas

Densidad: $8 / 0.5 \text{ KLOC} = 16 \text{ defectos/KLOC}$

Eficiencia: $8 \text{ defectos} / 2 \text{ horas} = 4 \text{ defectos/hora}$

Autoevaluaciones

Subtema 2.1 – *Simulación de Inspección Fagan*

- **Actividad:** Asignar roles (moderador, lector, autor, revisor) para revisar un fragmento de código vulnerable a SQL Injection. Registrar defectos usando un checklist basado en OWASP.
- **Criterios:**

Identificación correcta de defectos	60%
Aplicación de roles según Fagan	30%
Uso adecuado del checklist	10%

Subtema 2.2 – *Diseño de checklist para requisitos*

- **Actividad:** Crear checklist que verifique trazabilidad, claridad, ausencia de ambigüedades y criterios de aceptación en los requisitos de un sistema universitario.
- **Criterios:**

Cobertura de ítems críticos	70%
Claridad y utilidad	30%

Subtema 2.3 – Cálculo de métricas

Escenario:

- Artefacto: 1,200 LOC
- Defectos detectados: 15
- Defectos escapados post-revisión: 3
- Tiempo invertido: 6 horas

Actividad: Calcular:

- Densidad de defectos.
- Eficacia.
- Eficiencia.

Criterios:

<i>Criterio</i>	<i>%</i>
<i>Correctitud de cálculos</i>	80%
<i>Interpretación de resultados</i>	20%

Evaluaciónv– Plan de revisiones

Caso: Módulo de pagos en línea para matrículas estudiantiles, de alta criticidad por contener datos personales y transacciones.

Actividad:

- Diseñar un plan de revisiones que combine:
 - **Inspección Fagan** para código crítico (validación, cifrado, autenticación).
 - **Pair Programming** para desarrollo de endpoints.
 - **Walkthroughs** durante el análisis de requisitos.
- Incluir cronograma y métricas objetivo:
 - Densidad de defectos < 3 defectos/KLOC.
 - Eficacia $\geq 85\%$.

Criterios:

<i>Criterio</i>	<i>%</i>
<i>Adecuación técnica según riesgos</i>	40%
<i>Viabilidad del cronograma</i>	30%
<i>Pertinencia de métricas</i>	20%
<i>Claridad y estructura</i>	10%

Aseguramiento de la calidad

El Aseguramiento de la Calidad de Software (SQA, por sus siglas en inglés) es un enfoque sistemático que integra actividades, procesos y herramientas para garantizar que tanto los procesos de desarrollo como los productos finales cumplan con estándares internacionales, requisitos regulatorios y expectativas de los usuarios. En un entorno donde sistemas críticos (por ejemplo, aplicaciones de salud, financieras o académicas) manejan datos sensibles y requieren alta disponibilidad, el SQA se convierte en una pieza clave para mitigar riesgos, optimizar costos y fortalecer la confianza de clientes y audiencias.

3.1 Procesos SQA

Los procesos SQA abarcan desde la planificación y definición de políticas de calidad hasta la auditoría y el seguimiento de acciones correctivas. A continuación describimos dos elementos esenciales:

a) Auditorías SQA

Una auditoría SQA es una evaluación independiente, estructurada y documentada, que revisa el grado de cumplimiento de procesos, productos e infraestructura frente a estándares y normativas.

Tipos de auditoría

<i>Categoría</i>	<i>Objeto de evaluación</i>	<i>Ejemplo en Nicaragua</i>
Procesos	Adherencia a metodologías (CMMI, Scrum, ISO 9001)	Auditoría interna de un desarrollo ágil en una start-up tecnológica
Productos	Validación de artefactos (código, documentación, pruebas)	Revisión de la documentación de requisitos de un sistema universitario
Sistemas	Infraestructura, herramientas y entornos de desarrollo	Inspección de configuración de servidores que alojan una plataforma de telemedicina

Fases de la auditoría

1. Planificación

- Definición de alcance y criterios de éxito.
- Selección de auditores y responsables.

2. Ejecución

- Revisión de evidencias: registros de control de versiones, reportes de pruebas, actas de reuniones.
- Entrevistas a miembros del equipo (desarrolladores, testers, jefes de proyecto).

3. Informe

- Documentación de no conformidades con referencias a estándares (p. ej., ISO 19011:2018).
- Recomendaciones de mejora y nivel de severidad.

4. Seguimiento

- Verificación de la implementación de acciones correctivas.
- Cierre formal de hallazgos tras evidenciar las correcciones.

Ejemplo práctico: Una empresa nicaragüense de software de salud realiza una auditoría HIPAA para verificar que todas las comunicaciones de datos estén cifradas (TLS 1.2 o superior) y que existan registros de acceso con retención mínima de seis meses.

b) SQAP (Software Quality Assurance Plan)

El SQAP es el documento maestro que define cómo se implementará el SQA dentro de un proyecto. Es la hoja de ruta donde se especifican políticas, procedimientos, métricas, roles y responsabilidades.

Componentes esenciales del SQAP

<i>Componente</i>	<i>Contenido</i>	<i>Ejemplo local</i>
<i>Estándares</i>	Normas aplicables (ISO 25010, MISRA-C, PEP8, Ley 787 nicaragüense de protección de datos)	PEP8 para proyectos Python en la UNAN-Managua
<i>Actividades</i>	Revisiones formales e informales, auditorías, pruebas automatizadas	Inspecciones Fagan semanales en código crítico
<i>Métricas</i>	Indicadores de calidad (densidad de defectos, cobertura de pruebas, deuda técnica)	Objetivo: < 3 defectos/KLOC; cobertura ≥ 80 %
<i>Responsabilidades</i>	Roles y propietarios de tareas SQA (auditor líder, coordinador de métricas, testers)	Auditor líder: Ing. María Gómez
<i>Riesgos</i>	Identificación de riesgos y planes de mitigación (pérdida de datos, falta de recursos)	Backup diario, ambiente de pruebas aislado
<i>Cronograma</i>	Fechas clave para auditorías, revisiones y entregas	Primer SQAP: semana 2; auditoría de procesos: semana 6

Plantilla base según IEEE 730-2014

1. Propósito
2. Documentos de referencia
3. Gestión: roles y responsabilidades
4. Procesos de documentación
5. Estándares, prácticas y convenciones
6. Revisiones técnicas
7. Pruebas
8. Gestión de configuración
9. Plan de auditorías
10. Métricas y criterios de aceptación

3.2 Roles del Equipo SQA

Para cubrir de forma integral el aseguramiento de calidad, se forma un equipo multifuncional con responsabilidades claras:

<i>Rol</i>	<i>Responsabilidades clave</i>	<i>Habilidades requeridas</i>	<i>Ejemplo local</i>
<i>Auditor de procesos</i>	Verificar cumplimiento de metodologías (CMMI, ISO 9001)	Conocimiento profundo de CMMI e ISO 9001	Auditar proyecto de sistema ERP para MIFIC
<i>Especialista en métricas</i>	Diseñar, recolectar y analizar KPIs de calidad	Manejo de Grafana, Power BI, Excel avanzado	Reporte mensual de defectos/KLOC
<i>Responsable de cumplimiento</i>	Asegurar adherencia a regulaciones (GDPR, PCI-DSS, Ley 787)	Conocimiento legal y normativo	Cumplimiento PCI-DSS en fintech local
<i>Coordinador de pruebas</i>	Planificar y supervisar actividades de testing (unit, integración)	Herramientas de automatización (Selenium, JUnit)	Diseño de suite de pruebas para app médica
<i>Gestor de mejoras</i>	Implementar acciones correctivas y fomentar prácticas Lean/Kaizen	Técnicas de mejora continua, Kaizen, Lean	Talleres de retrospectiva y mejora de proceso

Ejemplo: En un proyecto bancario, el Responsable de cumplimiento garantiza que todas las transacciones cumplan con PCI-DSS, mientras el Auditor de procesos revisa la implementación de Scrum en el equipo de desarrollo.

3.3 Automatización en SQA

Mover las actividades de calidad “hacia la izquierda” (Shift Left) implica integrar validaciones automáticas lo más temprano posible en el ciclo de desarrollo, reduciendo retrabajos y acelerando la entrega.

a) SonarQube

Es una plataforma de análisis estático de código, soporta más de 30 lenguajes. Ofrece:

- **Detección de bugs** (p. ej., NullPointerException sin captura).
- **Vulnerabilidades** (p. ej., inyección SQL no mitigada).
- **Code smells** (código duplicado, métodos demasiado largos).
- **Métricas clave:** Fiabilidad, Seguridad, Deuda técnica (en días de corrección estimados).

Ejemplo de regla: S2589 — detecta comparaciones booleanas redundantes (if (flag == true)).

b) Checkstyle

Orientado a validar estándares de codificación en Java:

- Convenciones de nombres (clases, métodos, variables).
- Tamaño máximo de métodos y líneas.
- Consistencia de espaciado y sangrías.

Ejemplo de regla: AvoidStarImport — prohíbe import java.util.*.

c) Otras herramientas habituales

<i>Herramienta</i>	<i>Tipo</i>	<i>Caso de uso</i>
PMD	Análisis estático	Detectar recursos no cerrados, variables no usadas
ESLint	Linter JavaScript	Forzar uso de === en lugar de ==
FindBugs	Análisis bytecode Java	Identificar posibles deadlocks o errores comunes

d) Flujo de automatización típico

A[Commit de código] --> B[CI/CD Pipeline]

B --> C{Análisis estático}

C --> |Aprobado| D[Despliegue a staging]

C --> |Falla| E[Notificación al equipo]

En este flujo, cada commit dispara el pipeline que ejecuta SonarQube, Checkstyle o ESLint. Si cumple las reglas, se despliega en un entorno de pruebas; si no, se notifica para corregir.

Autoevaluación

Subtema 3.1 – Identificar elementos faltantes en SQAP

- **Actividad:** Analizar un extracto de SQAP con omisiones (p. ej., ausencia de métricas, cronograma de auditorías, gestión de riesgos). Identificar 5 elementos críticos ausentes según IEEE 730.
- **Criterios:**

<i>Criterio</i>	<i>%</i>
<i>Precisión en identificación</i>	70%
<i>Referencia a estándares IEEE</i>	30%

Subtema 3.2 – Diagrama de responsabilidades

- **Actividad:** Diseñar un organigrama para un equipo SQA de 5 miembros, vinculando roles con tareas específicas (p. ej., Especialista en métricas → elaboración de reportes KPIs mensuales).
- **Criterios:**

<i>Criterio</i>	<i>%</i>
<i>Claridad en la estructura</i>	40%
<i>Asignación lógica de tareas</i>	40%
<i>Profesionalismo visual</i>	20%

Subtema 3.3 – Interpretar informe SonarQube

- **Actividad:** Dado un reporte con 20 bugs (5 críticos), cobertura de pruebas 65 %, deuda técnica 8 días y 3 vulnerabilidades (1 alta), priorizar acciones correctivas y justificar la estrategia.
- **Criterios:**

<i>Criterio</i>	<i>%</i>
<i>Análisis crítico de métricas</i>	60%
<i>Plan de acción viable</i>	40%

Evaluación Auditoría Simulada

Caso de estudio: Proyecto fintech que incluye:

- Especificación de requisitos v1.4 (según IEEE 830).
- Plan de pruebas v2.1.
- Reporte de métricas del Q2-2024.

Actividades:

1. Verificación de trazabilidad

- ¿Cada requisito tiene un caso de prueba asociado?
- *Ejemplo de hallazgo:* RQ-07 carece de prueba unitaria.

2. Cumplimiento de estándares

- ¿Los documentos siguen la plantilla IEEE 830?
- *Ejemplo de hallazgo:* El plan de pruebas no incluye sección “Criterios de salida”.

3. Consistencia de métricas

- ¿Los valores reportados son coherentes (p.ej., densidad de defectos en pruebas vs. producción)?
- *Ejemplo de hallazgo:* Informe Q2 muestra 2.1/KLOC en pruebas y 5.3/KLOC en producción sin explicación de brecha.

Criterios de evaluación:

Criterio	%
Detección de no conformidades	50%
Fundamentación técnica de hallazgos	30%
Propuestas de mejora y seguimiento	20%

Estrategias de pruebas

En la Ingeniería de Software, las pruebas son la última línea de defensa contra la fuga de defectos hacia producción. Siguiendo modelos como el “V-Model” (Myers, 2004), las actividades de prueba se planifican en capas que corresponden a cada fase de desarrollo, de modo que los errores se detecten lo más temprano posible, reduciendo exponencialmente el costo de corrección y el riesgo para el negocio.

4.1 Niveles de Pruebas

El “**V-Model**” alinea cada fase de desarrollo con su contraparte en pruebas. Desde la unidad más pequeña (función o clase) hasta la validación por el usuario, cada nivel tiene objetivo, técnicas y métricas propias.

a) Pruebas Unitarias (Componentes)

Propósito. Verificar la lógica interna de las unidades de software (funciones, métodos, clases) de forma aislada.

- **Herramientas típicas:**
 - **Java:** JUnit (@Test void validarCorreo()), TestNG.
 - **C#:** NUnit, MSTest.
 - **Python:** pytest, unittest.
- **Métricas clave:**
 - **Cobertura de código:** porcentaje de líneas o ramas ejecutadas.
 - **Número de aserciones por prueba:** medir la exhaustividad.
 - **Tamaño de los casos:** mantener cada prueba enfocada en un único comportamiento.

- **Ejemplo contextualizado (Nicaragua):**

Una función en Python que convierte montos de córdobas a dólares con la tasa del día (por ejemplo, 35.25 NIO/USD). La prueba unitaria debe analizar distintos escenarios:

- **Tasa válida** (35.25) y monto positivo → resultado correcto.
 - **Monto cero** → retorna cero.
 - **Monto negativo** → lanza excepción ValueError.
- **Fundamento:** El desarrollo guiado por pruebas (TDD, Beck 2003) recomienda escribir primero la prueba, asegurando que cada unidad cumpla su contrato antes de integrarse.

b) Pruebas de Integración

Propósito. Validar la interacción y correcta colaboración entre módulos o subsistemas.

- **Patrones de integración:**

1. **Big Bang:** se ensamblan todos los componentes de una sola vez (alto riesgo).
2. **Incremental:**
 - *Top-down:* primero los módulos de alto nivel, luego los subordinados.
 - *Bottom-up:* primero las librerías y módulos base, luego la capa superior.
 - *Sandwich:* combina ambas direcciones.

- **Métricas y criterios:**

1. Número de interfaces probadas.
2. Incidencias por tipo de mensaje entre servicios (SOAP/REST).
3. Tiempo de respuesta agregado en el flujo.

- **Ejemplo real:**

En una pasarela de pago de una tienda en línea nicaragüense, se debe probar la comunicación REST entre el microservicio de autorización de tarjetas y el módulo de facturación:

1. Envío de JSON con campos { tarjeta, cvv, monto }.
2. Validar respuesta 200 OK y {"transaccion":"aprobada"}.
3. Probar formatos de fecha distintos, p. ej. ISO 8601 vs. DD/MM/YYYY, para evitar errores de parseo (caso real en integración PayPal–SAP).

c) Pruebas de Sistema

Propósito. Verificar el comportamiento del sistema completo en un entorno lo más parecido posible al de producción.

- **Tipos:**

- **Funcionales:** comprueban que el software cumpla los requisitos documentados (casos de uso, historias de usuario).
- **Regresivas:** re-ejecución de pruebas tras un cambio para asegurar que no se introdujeron nuevos errores.
- **De humo (smoke tests):** validaciones rápidas de rutas críticas.

- **Ejemplo:**

Flujo completo de comercio electrónico para productos artesanales de Masaya:

- Búsqueda por categoría “cerámica”.
- Añadir al carrito.
- Proceso de pago con tarjeta local.
- Generación de comprobante y notificación por correo.

d) Pruebas de Aceptación (UAT – User Acceptance Testing)

Propósito. Validación final por parte de usuarios o clientes, garantizando que el sistema satisface sus necesidades de negocio.

- **Variantes:**
 - **Alpha:** en entorno controlado, con usuarios internos o early adopters.
 - **Beta:** con usuarios reales en producción limitada (ej.: 500 estudiantes prueban la app de matrícula antes del lanzamiento oficial).
- **Criterio de éxito:** “El sistema satisface las condiciones de negocio y criterios de aceptación”, definidos en requisitos.

4.2 Técnicas de Diseño de Pruebas

Para diseñar casos de prueba efectivos, se emplean técnicas basadas en la especificación (caja negra) y en la estructura interna del código (caja blanca).

a) Caja Negra (Especificación)

<i>Técnica</i>	<i>Descripción</i>	<i>Ejemplo: formulario de login</i>
Partición	Agrupar entradas que deberían comportarse igual.	Usuario válido vs. usuario no registrado.
Equivalencia		
Valores Límite	Prueba en los extremos del dominio de entrada.	Usuario: vacío, 1 carácter, 50 caracteres.
Tablas de Decisión	Combina condiciones de entrada en una tabla lógica.	2FA activo + credenciales correctas → OK.

Caso límite crítico (Gherkin):

Escenario: Longitud máxima de usuario

Dado que el campo “usuario” acepta hasta 50 caracteres

Cuando ingreso "A".repeat(50)

Entonces el sistema permite el acceso con credenciales válidas

b) Caja Blanca (Estructural)

Se basa en la estructura interna del código:

- **Cobertura de sentencias:** cubrir el 100 % de líneas ejecutadas.
- **Cobertura de ramas:** cada if/else, switch debe evaluarse en ambos sentidos.
- **Cobertura de caminos:** garantizar que todos los flujos posibles se prueban (McCabe, 1976).
- **Herramientas:**
 - **JaCoCo** (Java).
 - **Coverage.py** (Python).
 - **dotCover** (C#).

4.3 Pruebas No Funcionales

Más allá de la lógica, el sistema debe cumplir atributos de calidad definidos en ISO 25010. A continuación, los tipos más críticos:

TIPO	OBJETIVO	HERRAMIENTAS	MÉTRICAS CLAVE
RENDIMIENTO	Evaluar escalabilidad y tiempos de respuesta	JMeter, k6	p90 < 2 s en 1,000 usuarios concurrentes
SEGURIDAD	Detectar vulnerabilidades	OWASP ZAP, Burp Suite	Número de vulnerabilidades CVSS > 7
USABILIDAD	Medir experiencia de usuario	Hotjar, UserTesting	Tasa de éxito de tareas > 90 %
COMPATIBILIDAD	Verificar multi-entorno y dispositivos	BrowserStack, Sauce Labs	% de navegadores/dispositivos compatibles

- **Ejemplo de prueba de seguridad (Python):**

```
# Test SQL Injection en función de login  
payload = "' OR '1'='1"  
response = login(username=payload, password="cualquiera")  
assert "Error de autenticación" in response
```

Autoevaluación

Subtema 4.1 – Clasificar Casos por Nivel de Prueba

- **Actividad:** Dada la lista de escenarios, asignarles el nivel adecuado (Unitario, Integración, Sistema, Aceptación).
- **Escenarios:**
 1. “Validar respuesta API /pagos con JUnit” → Unitarias
 2. “Probar interacción módulo inventario-ventas” → Integración
 3. “Flujo completo de reserva hotel” → Sistema
 4. “Cliente revisa reportes contables” → Aceptación
- **Criterio:** 100 % correcta clasificación.

Subtema 4.2 – Diseño de Casos con Valores Límite

- **Actividad:** Para el campo “edad” (rango 18–99), definir casos de prueba:
 - 18 (mínimo)
 - 99 (máximo)
 - 17 (bajo mínimo)
 - 100 (sobre máximo)
 - Campo vacío
- **Criterios:**
 - **Cobertura de límites** (70 %)
 - **Precisión técnica** (30 %)

Subtema 4.3 – Selección de Herramientas

- **Actividad:** Asociar el escenario al conjunto de herramientas más adecuado:
 1. “Prueba de carga de 10,000 usuarios concurrentes” → JMeter/k6
 2. “Detectar XSS en formulario web” → OWASP ZAP
 3. “Validar UI en iOS/Android” → BrowserStack
- **Criterio:** 100 % elección adecuada.

Evaluación Sumativa Tema 4 – Plan de Pruebas para Microservicio

Contexto: Microservicio de autenticación basado en JWT para una plataforma de matrícula en línea.

<i>Nivel</i>	<i>Caso de Prueba</i>	<i>Técnica</i>	<i>Datos</i>	<i>Resultado Esperado</i>
Unitario	Validar formato de email	Caja Blanca	test@dom.com	true (válido)
Unitario	Generación de JWT expirado	Caja Negra	exp=timestamp_pasado	Token rechazado
Integración	Consumo API de autorización	Valores Límite	Rol="superadmin"	Acceso concedido
Sistema	Carga de 100 usuarios/s	Rendimiento	Script JMeter	Latencia < 2 s (p95)
Seguridad	SQL Injection en login	Caja Negra	' OR '1'='1	Error 400 controlado
Compatibilidad	UI en Chrome, Firefox, Edge	Combinatoria	Versiones recientes	Interfaz consistente

Criterios de evaluación:

<i>Criterio</i>	<i>%</i>
<i>Cobertura de niveles críticos</i>	30%
<i>Adecuación técnica de las técnicas</i>	30%
<i>Calidad y realismo de datos de prueba</i>	20%
<i>Claridad de resultados esperados</i>	20%

Bibliografía y Webgrafía

- Beck, K. (2003). *Test-Driven Development: By Example*. Addison-Wesley.
- ISTQB. (2021). *Certified Tester Foundation Level Syllabus* (Version 4.0). International Software Testing Qualifications Board.
- Kaner, C., Bach, J., & Pettichord, B. (2008). *Lessons Learned in Software Testing: A Context-Driven Approach*. Wiley.
- Myers, G. J., Sandler, C., & Badgett, T. (2004). *The Art of Software Testing* (2nd ed.). Wiley.
- ISO/IEC 25010:2011. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.
- McCabe, T. J. (1976). *A Complexity Measure*. IEEE Transactions on Software Engineering, SE-2(4), 308–320.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- OWASP Foundation. (2021). *OWASP Testing Guide v4*. Open Web Application Security Project

Alpha Testing: Pruebas de aceptación realizadas en un entorno controlado por el equipo de desarrollo antes de liberar el software a usuarios externos.

Beta Testing: Pruebas de aceptación realizadas por usuarios reales en un entorno de producción o cercano, para detectar defectos no encontrados internamente.

Big Bang (Integración): Estrategia de pruebas de integración en la que todos los módulos se combinan y se prueban de una vez, incrementando el riesgo de errores difíciles de aislar.

Caja Blanca (White-box Testing): Técnicas de prueba que utilizan el conocimiento de la estructura interna del código para diseñar casos, por ejemplo, cobertura de sentencias y ramas.

Caja Negra (Black-box Testing): Técnicas de prueba basadas en los requisitos o especificaciones, sin conocer la implementación interna. Ejemplos: partición de equivalencia, valores límite.

Cobertura de Código: Métrica que mide el porcentaje del código fuente ejecutado por un conjunto de pruebas; niveles óptimos suelen superar el 80%.

Compatibilidad (Testing): Pruebas que validan el correcto funcionamiento del software en diferentes dispositivos, sistemas operativos o navegadores.

Datos de Prueba: Conjunto de valores o entradas utilizados para ejecutar casos de prueba y evaluar comportamientos del software.

Gherkin: Lenguaje de especificación estructurado en pasos ("Dado", "Cuando", "Entonces") usado en BDD (Behavior Driven Development).

Integración Incremental: Estrategias top-down, bottom-up o sandwich que integran módulos gradualmente para facilitar la detección de errores.

ISO 25010: Estándar internacional que define un modelo de calidad del software incluyendo características como rendimiento, seguridad, compatibilidad y usabilidad.

Junit: Framework de pruebas unitarias para Java ampliamente usado para automatizar la verificación de métodos y clases.

JWT (JSON Web Token): Estándar abierto para intercambio seguro de información, utilizado frecuentemente en autenticación.

McCabe Complexity: Métrica que mide la complejidad ciclomatica de un programa, basada en el número de caminos lógicamente independientes.

Microservicio: Arquitectura de software en la que una aplicación está compuesta por pequeños servicios independientes que se comunican a través de APIs.

Partición de Equivalencia: Técnica de caja negra que agrupa entradas en clases equivalentes, reduciendo el número de casos de prueba necesarios.

Pruebas de Aceptación (UAT): Último nivel de prueba donde el cliente o usuario valida que el sistema satisface sus necesidades de negocio.

Pruebas de Integración: Validan la correcta interacción entre componentes o módulos previamente probados de manera individual.

Pruebas de Sistema: Prueban el sistema completo como una unidad, asegurando el cumplimiento de requisitos funcionales y no funcionales.

Pruebas Unitarias: Verifican el funcionamiento correcto de las unidades más pequeñas del código (por ejemplo, funciones o clases).

Pytest: Framework de pruebas para Python que permite escribir pruebas unitarias y de integración de forma concisa.

SQL Injection: Técnica de ataque de seguridad que inserta código SQL malicioso en campos de entrada para manipular bases de datos.

Valores Límite: Técnica de prueba de caja negra que se centra en los extremos de los rangos de entrada donde suelen concentrarse los defectos.

V-Model: Modelo de desarrollo de software que asocia fases de construcción con fases de pruebas correspondientes para detectar defectos tempranamente.